

Related

[Your basic int: a most powerful data type](#)

The int data type is generic, effective, simple and efficient. It can represent any digital data. Furthermore, an int can point into any type of array. That's as generic as it gets.

[yourbasic.org](#)

[Numbers that start with zero](#)

Octal literals start with 0, hexadecimal with 0x, and decimal literals never start with zero.

[yourbasic.org](#)

Most Read



[Do you make these Go coding mistakes?](#)

[Why Go? – Key advantages you may have overlooked](#)

[Go string handling overview \[cheat sheet\]](#)

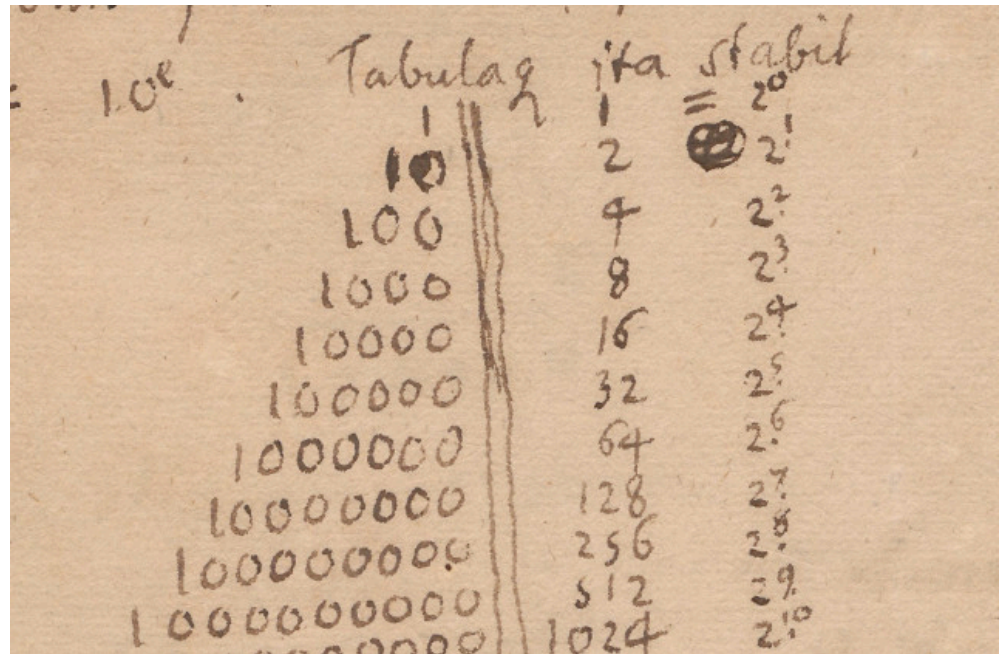
[Type, value and equality of interfaces](#)

[Concurrent programming](#)

[See all 178 Go articles](#)

Bitwise operators [cheat sheet]

[yourbasic.org/golang](#)



Gottfried Wilhelm Leibniz, 1697

- » [Number literals](#)
- » [Built-in operators](#)
- » [Package math/bits](#)
- » [Bit manipulation code example](#)

Number literals

The binary number 10000_2 can be written as 020, 16 or 0x10 in Go.

Literal	Base	Note
020	8	Starts with 0
16	10	Never starts with 0
0x10	16	Starts with 0x

Built-in operators

Operation	Result	Description
0011 & 0101	0001	Bitwise AND
0011 0101	0111	Bitwise OR

Operation	Result	Description
<code>0011 ^ 0101</code>	0110	Bitwise XOR
<code>^0101</code>	1010	Bitwise NOT (same as <code>1111 ^ 0101</code>)
<code>0011 &^ 0101</code>	0010	Bitclear (AND NOT)
<code>00110101<<2</code>	11010100	Left shift
<code>00110101<<100</code>	00000000	No upper limit on shift count
<code>00110101>>2</code>	00001101	Right shift

- The binary numbers in the examples are for explanation only. Integer literals in Go must be specified in octal, decimal or hexadecimal.
- The bitwise operators take both signed and unsigned integers as input. The right-hand side of a shift operator, however, must be an unsigned integer.
- Shift operators implement arithmetic shifts if the left operand is a signed integer and logical shifts if it is an unsigned integer.

Package `math/bits`

Operation	Result	Description
<code>bits.UintSize</code>	32 or 64	Size of a uint in bits
<code>bits.OnesCount8(00101110)</code>	4	Number of one bits (population count)
<code>bits.Len8(00101110)</code>	6	Bits required to represent number
<code>bits.Len8(00000000)</code>	0	
<code>bits.LeadingZeros8(00101110)</code>	2	Number of leading zero bits
<code>bits.LeadingZeros8(00000000)</code>	8	
<code>bits.TrailingZeros8(00101110)</code>	1	Number of trailing zero bits
<code>bits.TrailingZeros8(00000000)</code>	8	
<code>bits.RotateLeft8(00101110, 3)</code>	01110001	The value rotated left by 3 bits
<code>bits.RotateLeft8(00101110, -3)</code>	11000101	The value rotated right by 3 bits
<code>bits.Reverse8(00101110)</code>	01110100	Bits in reversed order
<code>bits.ReverseBytes16(0x00ff)</code>	0xff00	Bytes in reversed order

- The functions operate on **unsigned integers**.
- They come in different forms that take arguments of different sizes. For example, `Len`, `Len8`, `Len16`, `Len32`, and `Len64` apply to the types `uint`, `uint8`, `uint16`, `uint32`, and `uint64`, respectively.

- The functions are recognized by the compiler and on most architectures they are treated as [intrinsics](#) for additional performance.

Bit manipulation code example



[Bitmasks, bitsets and flags](#) shows how to implement a bitmask, a small set of booleans, often called flags, represented by the bits in a single number.

This work is licensed under a [CC BY 3.0 license](#).